

1 – GETTING STARTED

This chapter introduces the fundamentals needed to begin working with Python. You will learn how to write and run Python programs using different tools and environments, ranging from interactive sessions to full program execution. Along the way, you'll explore how Python displays output, accepts input from the keyboard, and uses variables to store data.

You'll also be introduced to comments, naming conventions, and a handful of built-in functions that help you understand and explore Python code. By the end of this chapter, you will have the foundation needed to comfortably experiment with Python, read simple programs, and prepare for more advanced topics later in the course.

Learning outcomes

By the end of this chapter, you will be able to:

1. Run Python code in Jupyter Notebooks
2. Display output with `print()` and read keyboard input with `input()`
3. Write comments that add value rather than noise
4. Create Variable that follow Python's rules and this course conventions
5. Use `id()`, `type()`, `dir()`, `help()`, and `eval()` to inspect Python objects

1.a – Writing and Running Code

There are several common ways to run Python code. Each useful in different situations:

- The Python REPL
- Interactive notebooks (VS Code, Jupyter, Google Colab and PyCharm)
- Running a script from the command line
- Executing a script as an executable file

Fun fact

A Pythonista is someone who enjoys using Python – whether they're a beginner, expert, or somewhere between.

Method	Best for	Main limitation
REPL	Quick experiments, testing one idea, debugging	Awkward for anything longer than a few lines
Command Line	Running a finished program	When the session ends, you cannot inspect anything afterwards

Command Line with -i	Debugging a finished program	Still requires a terminal
Executable script	Distributing to end users	Unifamily systems only
Notebook	Teaching, demonstrations, data work	Not a way to ship software

1.a.1 – Python REPL

The Python REPL is the simplest and most flexible ways to run Python code. It lets you write one line at a time and get immediate feedback, which makes it popular with both beginners and veteran *Pythonistas*.

To use the REPL, you start the python interpreter. This opens a command prompt similar to your operating system shells. This is a **Read-Evaluate-Print Loop**.

- Python **reads** your input
- **evaluates** it
- **prints** the result back to the screen
- and **loops** around to do it again

Note

In the REPL, Python prints the value of any expression automatically — you do not need `print()`. In a script, it does not. This surprises almost everyone at least once.

You can use this technique to incrementally build substantial python applications or parts of applications. You are able to examine each object at your leisure. You may also debug problematic code this way.

The downside is that it is not convenient for larger/longer block of code or distributing code to end users or when typing is challenging.

To leave the REPL, type `exit()` or press `Ctrl-Z` then `Enter` on Windows, or `Ctrl-D` on Unix, Linux, and macOS.

Below is a sample session in the python **repl** in a windows 10 environment:

```
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
PS C:\progs\new100\labs> python
```

```
Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep 5 2022, 14:08:36) [MSC v.1933 64 bit (AMD64)] on win32
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> 2 + 6
```

```
8
>>> 8 / 3
2.6666666666666665
>>>
>>> hi = 'hi '
>>> hi = hi * 4
>>> hi
'hi hi hi hi '
```

1.a.2 – Running Python code from the command prompt

Once your program is complete and behaving correctly (ideally 95-100% according to specifications), you typically run it from the command line.

```
«path_to_python» «name_of_code_file»

>>> python lec33_docstring.py
```

1.a.3 – Running Python code interactively from the command prompt

The above technique, terminates the python session after the python file is processed. This means you can't inspect variables or object afterwards.

If you use the `-i` switch, Python keeps the session alive. This allows you to examine variables, functions and objects created with the program.

```
«path_to_python» «name_of_code_file»

>>> python -i lec33_docstring.py
```

This is especially useful for debugging and exploration.

1.a.4 – Running Python as an Executable (Unix/Linux/macOS)

In this method of running python code there are two steps involved, changing the permission of the file and then inserting the path to the python interpreter in the first line of the code file.

```
chmod 755 «name_of_code_file»  
or  
chmod a+x «name_of_code_file»
```

And then modifying the code file, insert the following at the top of the file:

```
#!/«path_name_of_python_interpreter»
```

This is probably the best and the most professional way of running or distributing your code to everyone from the novice user to the seasoned developer.

This is one of the most professional ways to distribute Python programs to users of all experience levels.

Interesting fact

This approach works only on operating systems that support file permissions (Linux, Unix, macOS). On Windows, you can achieve a similar result using batch files or PowerShell scripts.

1.a.5 – Notebooks

Notebooks are quickly becoming one of the most popular ways to write, run and demonstrate Python code.

A notebook is made up of three types of cells:

- **Markdown cells** for text and explanations
- **Code cells** containing Python code
- **Output cells** that show the results of running code

Notebooks feels like browsing a live textbook: you can read the theory and see the code, run it, and immediately view the results – all in one place. You can also modify the code and re-run to observe changes.

Many online platforms provide pre-configured environments with the required libraries already installed. This is especially helpful for beginners or when working with complex setups like machine learning or image processing.

Popular notebook platform includes:

- Jupyter
- Binder
- Kaggle
- Google

- Microsoft
- CoCalc
- Datalore.

1.b – Sending output to the screen: using the `print()` function

The `print()` function displays output on the console. While it's simple to use, it also supports more advanced options that we will explore later in the semester.

```
>>> print(4)                                #displays the integer literal 4
4

>>> print(3 + 0.141592)                    #displays the float literal
3.141592

>>> print('Hello World')                   #displays the string literal
Hello World

>>> print(name)                             #display the contents of variable name
Narendra Pershad
```

1.c – Getting input from the keyboard: using the `input()` function

The `input()` function displays a prompt and waits for the user to type something. The results is always returned as a string, so it's usually stored in a variable for later use.

```
>>> name = input("What's your name? ")
What's your name? Narendra                #user response is shown in bold face

>>> print(name)(my_list)
Narendra
```

In the next chapter, we will learn how to convert this text input to numbers and other data types.

1.d – Comments

Comments are lines of text that are ignored by the interpreter. They are used to improve the readability of the code for coders themselves or other developers. For single-line comments, use the “#” symbol, for multi-line comments use either triple single or double quotes.

Fun fact

Comments are not written in the repl.

```
#this is a single-line comment

'''
This is a multi-line comment.
It can span more than
one line
'''
```

Because Python code is generally easy to read, avoid unnecessary comment that restates the obvious. Use comments sparingly to explain why something is done, and not what the code already makes clear

1.e – Variables

At the earliest days of computing, when computer scientist wants to use a piece of memory, they have to use a memory address to refer to that location, this is both tedious and error-prone. Shortly afterwards, language designers allow programmers to assign a name to a memory location. So instead of referring to numeric memory addresses the programmer uses descriptive names to manipulate values.

A variable gives a name to a piece of data (or location of that data). Like a box with a label, you may access what's inside the box if you know the name of the box.

```
name = 'Narendra'
age = 12
```

Once a variable is created in the REPL, it remains until the current session is destroyed. You can remove a variable using the `del` keyword.

1.e.1 – Rules for Naming Variable (enforced by Python)

- Variable must start with a letter or underscore

- Can not contain spaces
- Cannot be a python keyword
- Cannot contain include symbols like '!', '@', '\$', '#', or %'
- Avoid using name of built-in function 'list', 'str', 'sum', 'int', or 'len'.
- Avoid starting name with double underscores.

Interesting fact

The following will list all of python keywords:

```
help('keywords')
```

```
>>> student_1 = 'Justin Trudeau'
>>> list = [3]                                #legal, but very bad idea
>>> first_name = 'Narendra'                #not legal
```

1.e.2 –Naming Conventions (required for this course)

These are not enforced by Python, They are enforced by your instructor, and by many professional team that you might be a part of.

Fun fact

C# uses camelcase for variables and pascalcase for methods.

- Use descriptive names
- Avoid single letter unless they are meaningful in the context such as the body of a loop.
- Use **snake-casing** for variables
- Use **all caps** for constants.
- Avoid name that shadow built-in function : list, str, sum, int, len, type, id
- Avoid starting variables with single or double underscores.

```
>>> first_name = 'Narendra'

>>> GST = 0.14                                #suggest to the programmer GST is a
constant
>>> __first_name = 'Narendra'                 #legal but not recommended
>>> a = 3                                       #legal but not descriptive
```

1.f – Some useful built-in functions

Python includes many built-in functions that are available without any setup. These functions solve common problems and save time.

FYI

We will cover function formally later in chapter 5.

In addition to `print()` and `input()` here are some frequently used ones.

1.f.1 – `id()`

The `id()` function returns a unique identifier for an object. In CPython, this usually corresponds to the object's memory address.

The following code illustrates that `int` are immutable and `list` are mutable.

```
>>> a = 3
>>> id(a)
1275421851952                                #address of the int a

>>> a = a + 7
>>> id(a)
1275421852176                                #address of the int a (different)
                                           # ask your instructor to explain this

>>> b = []
>>> id(b)
1275429377408                                #address of the list b

>>> b.append(3)                               # adds an item to the list
>>> b
[3]

>>> id(b)
1275429377408                                #address of the list b (identical)
                                           # ask your instructor to explain this
```

1.f.2 – `type()`

The `type()` function tells you the data type of an object:

FYI

We will cover datatypes in the next chapter.

```
>>> type(3)
<class 'int'>

>>> type(3.1415)
<class 'float'>

>>> type('Narendra')
<class 'str'>

>>> type(True)
<class 'bool'>

>>> type([])
<class 'list'>

>>> type((4,))
<class 'tuple'>

>>> type({'a'})
<class 'set'>
```

1.f.3 – dir()

The `dir()` function shows all the objects that are currently in scope. The first usage shows the objects in the global scope and the second usage show the objects in the `__builtins__` namespace.

FYI

A more powerful way to explore Python's built-in documentation system is to use the command:

`python -m pydoc -b`

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'a', 'b']
```

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
...
'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super',
'tuple', 'type', 'vars', 'zip']
```

This function is very useful for introspection. It allows you to list the attributes and methods of an object especially in the repl.

1.f.4 – help()

The `help()` function launches Python built-in help system. Invoking the `help()` function with `str` and `str.split` arguments.

```
>>> help(str)
Help on class str in module builtins:

class str(object)
 | str(object='') -> str
 | str(bytes_or_buffer[, encoding[, errors]]) -> str
 |
 | Create a new string object from the given object. If encoding or
 | errors is specified, then the object must expose a data buffer
```

```
>>> help(str.split)
Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1)
    Return a list of the substrings in the string, using sep as the
    separator string.

    sep
        The separator used to split the string.
```

It provides interactive access to Python's documentation. It is useful for quickly getting information about modules, classes, functions, keywords and other Python objects.

1.f.5 – eval()

The `eval()` function evaluates strings as Python code and returns the result.

Warning

Never call `eval()` on input you did not write yourself.

While it can be useful in controlled situations, **never use `eval()` on untrusted input**. It can execute malicious code and should be avoided in production programs.

```
>>> eval('2 + 3')
5

>>> list_string = "[1, 2, 3, 'hello', 4.5]"
>>> my_list = eval(list_string)

>>> print(my_list)
[1, 2, 3, 'hello', 4.5]

>>> print(type(my_list))
```

1.g – Summary

In this chapter, we explored the basics of getting started with Python and the different ways Python code can be written and executed. We learned that Python programs can be run using the REPL, from the command line, as executable scripts on Unix-based systems, or through interactive notebooks such as Jupyter and Google Colab. Each approach serves a different purpose, from quick experimentation to professional code distribution.

We introduced the Python REPL as an interactive environment that allows immediate feedback, making it ideal for learning, testing, and debugging small pieces of code. We also saw how Python scripts can be executed from the command prompt, including the use of the `-i` switch to keep the session alive for further inspection.

The chapter covered basic input and output using the `print()` and `input()` functions, emphasizing that user input is always received as text. We discussed comments and their role in improving code readability, along with best practices for using them effectively.

We examined variables as named references to data, reviewed the rules and conventions for naming them, and introduced the idea of constants. Finally, we explored several useful built-in functions such as `id()`, `type()`, `dir()`, and `help()`, which help with understanding objects, data types, and Python's built-in features.

By the end of this chapter, you should be comfortable running Python code in multiple environments, displaying output, accepting user input, working with variables, and using basic built-in tools to explore and understand Python programs.

1.h – Non coding exercises

1. What does REPL stand for, and what are the four things it does?
2. Name two advantages and two disadvantages of using the REPL.
3. What does the `-i` switch do when running a Python script from the command line?
4. What two steps are required to run a Python file as an executable on Unix/Linux/macOS?
5. What are the three types of cells in a notebook, and what is each used for?
6. What does `print()` do?
7. What data type does `input()` always return?
8. What is a comment, and what symbol starts a single-line comment?
9. What is a variable?
10. List three rules for naming a variable in Python.
11. What naming convention does this course use for variables and for constants?
12. What does `type()` do?
13. What does `id()` return?
14. What does `dir()` do?
15. What does `help()` do?
16. What does `eval()` do?
17. Why is `list = [3]` considered illegal or bad practice?